

Strings, Character Arrays and std::string in C++

[DEF] Definition to Remember

Strings in C++ can be represented as null-terminated C-style character arrays or dynamic std::string objects from the STL offering rich string manipulation methods and automatic memory management.

1. Concept Overview & Fundamentals

Strings, Character Arrays and std::string in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> push_back, contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

* Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

Problem 1: break keyword loop

Problem Description & Objective:

C++ Code Implementation:

Implem

```

/*
 * Que
 * Cre
 "exit"
 */
/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */
#include
#include

```

```
// Use
using

//
int ma
st
//
co
//
wh

}

//
re
}
```

Problem 2: date format

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * For
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */
```

```
#inclu
#inclu

// Use
using

//
int ma
in

//
co
//
co

//
re
}
```

Problem 3: fgets puts

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Rea
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#inclu
#inclu

// Use
using

//
int ma
st
```

```
//  
co  
//  
ge  
  
//  
co  
//  
re  
}
```

Problem 4: login system

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
#inclu  
  
// Use  
using  
  
//  
int ma  
co  
co  
  
st  
  
//  
co  
//  
co
```

```
//
ci

//
co
//
ci

//
if

//
}

}

//
re
}
```

Problem 5: password checker

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include
#include

// Use
using

//
int ma
co
```

```
st
do

//
}

//
co

//
re
}
```

Problem 6: reverse string function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wri
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
#include
#include

// Use
using
```

```
void r
    in
    in
    //
    wh

}

//
int ma
st
//
co
//
ge

re

//
co

//
re
}
```

Problem 7: string palindrome

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ===
 */

#include
#include
```

```
#inclu

// Use
using

bool i
    in
    in

//
wh

}
//
re
}

//
int ma
st
//
cc
//
ci
//
if

//
}

}

//
re
}
```

Problem 8: tic tac toe board

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Use
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
```

```
*
* 2.
*
*
* 3.
*
*
* 4
* *
*
*
*
*
* ==
*/

#include
#include

// Use
using

// Dec
void c
//
cc
//
fo

}
}

//
int ma
//
ve

};

//
cc
di

bc
bc
bc

//
cc
di

//
re
```

}

Problem 9: trim string

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Imp
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
#include

// Use
using

string
si
//
if
si
//
re
}

//
int ma
st

//
co
st
//
co

//
re
}
```

Problem 10: uppercase string

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wri
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include
#include
#include

// Use
using

//
int ma
st
//
co
//
ge

//
fo

}

//
co

//
re
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. `std::string` manages its own dynamic buffer, eliminating buffer overflow risks.
2. String operators: `+` for concatenation, `==` and `<` for lexicographical comparisons.
3. Useful member functions: `length()`, `substr()`, `find()`, `append()`, `replace()`, `c_str()`.
4. Two-pointer string reversal and palindrome checks operate in $O(N)$ time and $O(1)$ space.
5. Converting C++ string to C-style string: `str.c_str()` returns `const char*` pointer.

[RECALL] Quick Recall

`std::string` -> Dynamic Heap Buffer -> `+` Concatenation -> `substr/find` -> Two-Pointer Palindrome